

110.6. Devant, derrière : le buisson, teindre(..., DEVANT)

Sur Game Boy Color, une case du décor peut passer devant les lutins : `teindre(c, l, 2 | DEVANT)`. Deux buissons pareils : le héros passe devant le premier, derrière le second.

1. Ce qu'il faut comprendre

DERRIERE cache le héros derrière **TOUT le décor**. Souvent, on veut l'inverse : le héros devant l'herbe, mais derrière **un** buisson. Il faut alors marquer **les cases** qui passent devant, pas le lutin.

Ce qui est nouveau ici : **DEVANT**, avec `teindre()` (**Game Boy Color**). `teindre(12, 8, 2 | DEVANT)` met la case (12, 8) en palette 2 **et** la fait passer devant les lutins. La barre `|` réunit les deux réglages en un seul nombre : la palette dans les bits du bas, **DEVANT** dans le bit du haut (128). $2 | 128 = 130$.

La preuve par deux : les deux buissons ont le **même dessin** et la **même palette**. Seul le second a **DEVANT**. Le héros passe devant le premier, derrière le second.

La même règle que pour DERRIERE : la case ne cache le lutin que là où elle est **dessinée** (1, 2 ou 3). C'est pourquoi le buisson n'a **aucun 0** : avec des coins vides, deux touffes côte à côte laisseraient un petit trou entre elles, et l'on verrait un morceau du héros y rester figé pendant qu'il passe derrière.

Pourquoi la Color seulement ? Sur la Game Boy d'origine, il n'y a qu'une carte du décor, sans place pour ce réglage. La Color a une **seconde carte** (celle des palettes, le chapitre 16) : c'est là que se range **DEVANT**.

À toi : ajoute `| DEVANT` au premier buisson aussi : le héros passe derrière les deux.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <couleurFond>` → [0.88.3. La fonction couleurFond\(\) — une couleur du décor](#)
- `#include <couleurLutin>` → [0.107.4. La fonction couleurLutin\(\) — une couleur des personnages](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <teindre>` → [0.89.1. La fonction teindre\(\) — une case dans une palette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <teindreLutin>` → [0.107.5. La fonction teindreLutin\(\) — un lutin dans une palette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **820 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage – la mise en route de la console
main()	346 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← ecran() – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), teindre() – trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	94 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	176 o	← CopierTuiles – 9 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert – la petite routine qui copie les lutins
"DEUX BUISSONS"	13 o	← texte() – un texte gravé

Lettres gravées : BDEINOSUX

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

couleurFond()	x 6	72 o sur place, dans main()
teindre()	x 2	48 o sur place, dans main() + AdresseCase (10 o)
couleurLutin()	x 3	36 o sur place, dans main()
poser()	x 2	26 o sur place, dans main() + AdresseCase (10 o)
sprite()	x 1	23 o sur place, dans main()
ecran()	x 2	21 o sur place, dans main() + AttendreVBlank (16 o)
teindreLutin()	x 1	16 o sur place, dans main()
texte()	x 1	11 o sur place, dans main() + EcrireTexte (10 o)
image()	x 1	6 o sur place, dans main() + AttendreImage (28 o)
images()	x 1	2 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME (Game Boy Color) : deux buissons PAREILS. Le héros avance tout
// seul : il passe DEVANT le premier, et DERRIÈRE le second.
//
// Ce qui est nouveau : DEVANT, avec teindre().
// teindre(c, 1, 2);          la case prend la palette 2 (comme au chapitre 16)
// teindre(c, 1, 2 | DEVANT); palette 2 ET la case passe devant les lutins
// La barre | réunit les deux réglages : 2 | DEVANT = 2 + 128 = 130.
```

```
#include <Tuile>           // un dessin de 8 × 8 pixels
#include <couleurFond>      // choisit une couleur d'une palette du fond
#include <couleurLutin>     // choisit une couleur d'une palette des lutins
#include <ecran>            // éteint ou rallume l'écran
#include <poser>            // pose une tuile sur une case du fond
#include <teindre>          // met une case du fond dans une palette
#include <texte>            // écrit un texte à l'écran
#include <sprite>           // place un lutin de 8 × 8 au pixel près
#include <teindreLutin>     // met un lutin dans une palette
```

2. UN DESSIN — BUISSON

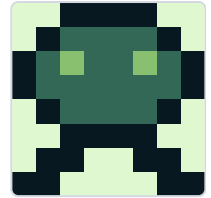
```
Tuile BUISSON = {          // Une touffe de feuilles : AUCUN 0, elle est toute pleine.
    "11222211",           // (des coins en 0 laisseraient un trou là où deux
    "12233221",           // touffes se touchent : on y verrait le héros)
    "22333322",
    "23333332",
    "23333332",
    "22333322",
    "12233221",
    "11222211",
};
```



à l'écran

3. UN DESSIN — HEROS

```
Tuile HEROS = { // 0 transparent, 1 clair, 2 moyen, 3 contour.
    "00333300",
    "03222230",
    "32122123",
    "32222223",
    "03222230",
    "00333300",
    "03300330",
    "33000333",
};
```



à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
    couleurFond(0, 0, 20, 28, 31); // palette 0 : le ciel (indice 0)...
    couleurFond(0, 3, 2, 4, 12); // ... et le texte (indice 3)
    couleurFond(2, 0, 20, 28, 31); // palette 2 : le ciel entre les feuilles,
    couleurFond(2, 1, 16, 30, 8); // vert clair,
    couleurFond(2, 2, 6, 22, 4); // vert,
    couleurFond(2, 3, 0, 10, 2); // vert sombre.
    couleurLutin(0, 1, 31, 28, 20); // le héros : rose clair,
    couleurLutin(0, 2, 31, 4, 2); // rouge,
    couleurLutin(0, 3, 10, 0, 0); // contour très sombre.

    ecran(0); // Dessin écran éteint.
    texte(1, 1, "DEUX BUISSONS");
    for (uint8_t l = 8; l < 10; l++) { // Lignes 8 et 9 :
        for (uint8_t c = 4; c < 6; c++) { // buisson 1, colonnes 4 et 5 :
            poser(c, l, BUISSON);
            teindre(c, l, 2); // palette 2, c'est tout.
        }
        for (uint8_t c = 12; c < 14; c++) { // buisson 2, colonnes 12 et 13 :
            poser(c, l, BUISSON);
            teindre(c, l, 2 | DEVANT); // palette 2, ET devant les lutins.
        }
    }
    ecran(1); // On rallume.

    uint8_t x = 0; // La place du héros, en pixels.

    while (true) { // La boucle du jeu :
        image();

        if (images() % 2 == 0) x++; // Un pixel toutes les 2 images.
        if (x > 152) x = 0;

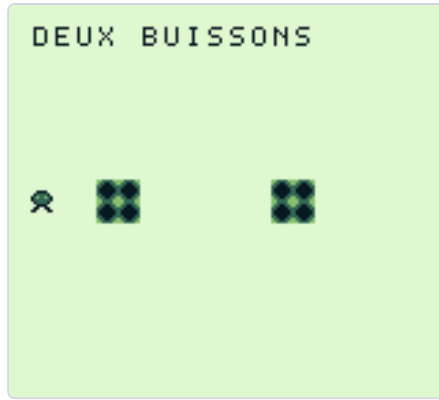
        sprite(0, x, 68, HEROS); // Pixel 68 : au milieu des buissons (lignes 8 et 9).
        teindreLutin(0, 0); // Après sprite() : sa palette (chapitre 16).
    }
}
```

3. Ce qui se passe, étape par étape

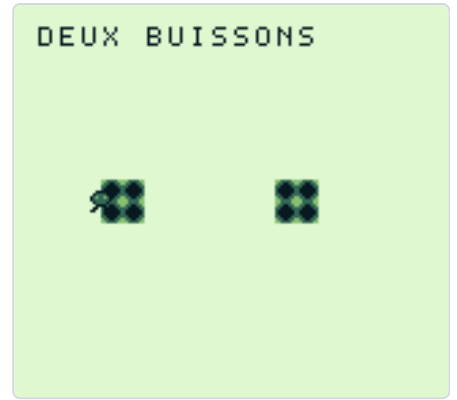
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



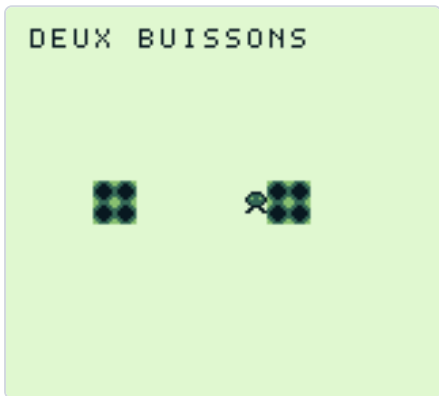
la toute première image — la console vient de commencer — image 1



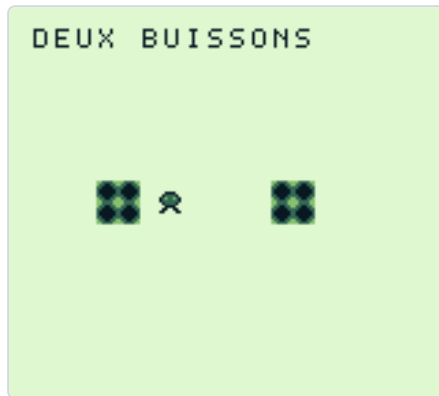
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Deux buissons verts identiques. Le héros rouge passe devant celui de gauche, et disparaît derrière celui de droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la cartouche est en couleur
- ✓ buisson 1 : le héros (rouge) est devant (31/4/2)
- ✓ buisson 2, DEVANT : on voit le buisson (vert) (6/22/4)

Ensuite : **110.7. Devant, derrière : le bateau passe sous le pont** — chapitre 17 : Le mouvement : les lutins au pixel près. · gameboy3 — écrire une cartouche Game Boy en C++